

Integrating Sentry into a C++ Project on Windows

A complete, replicable setup: native crash & error reporting, symbolicated stack traces with source context, automatic debug-symbol upload, and browser Session Replay — using CMake, MinGW-w64, the Sentry Native SDK, and sentry-cli.

Reference project: `sentryc++` — a from-scratch C++17 HTTP server.

Toolchain used: MinGW-w64 g++ 16.1.0 · CMake 3.31 · Sentry Native SDK 0.15.0 · sentry-cli 2.x · Sentry Browser SDK 8.9.0

Platform: Windows 11 (x86_64)

Replace all project-specific values (DSN, org slug, project id, auth token, release name) with your own when applying this to a new project.

Contents

1. Overview & mental model
2. Prerequisites & toolchain
3. Step 1 — Build the Sentry Native SDK
4. Step 2 — Wire Sentry into your CMake build
5. Step 3 — Initialise Sentry in code
6. Step 4 — Capturing errors (3 modes)
7. Step 5 — Upload debug symbols (the make-or-break step)
8. Step 6 — Session Replay (the "video")
9. Step 7 — Verify end-to-end
10. Troubleshooting matrix (the gotchas)
11. Security & housekeeping
12. Optional — link stack frames to your GitHub repo
13. Appendix A — command cheat sheet
14. Appendix B — final file listings

1 · Overview & mental model

Sentry for C++ has **two independent halves**. Getting "the full experience" means setting up both:

Half	SDK	Gives you	Runs in
Backend	Sentry Native SDK (<code>libsentry</code>)	Native crash capture (access violations, aborts), caught-exception reporting, manual events, symbolicated stack traces	Your compiled <code>.exe</code>
Frontend	Sentry Browser SDK (JavaScript)	Session Replay ("video"), browser error capture, distributed tracing	The web page served to the browser

The single most important concept: Sentry **symbolicates stack traces on its servers**, not on your machine. The raw crash event only contains memory addresses + a *debug-id*. To turn those addresses into `file:line` and show your source code, Sentry must already hold the **debug-information file** (and source bundle) whose debug-id matches the exact binary that crashed. **If you never upload them, you get blank/garbage traces.** This is the step everyone misses.

Three facts that follow from this, and that cost the most time to learn:

- **Re-upload after every rebuild.** Each build produces a new debug-id; an old upload won't match.
- **Build with `-g` and `-O0`.** No debug info → nothing to symbolicate. Optimization → wrong line numbers.
- **A message event has no stack trace unless you attach one.** Logging a string does not record "where".

2 · Prerequisites & toolchain

Tool	Why	Notes / version used here
MinGW-w64 (<code>g++</code>)	Compiler + linker	<code>g++ 16.1.0</code> at <code>C:\mingw64\bin</code> . Use a 64-bit build; keep one MinGW on PATH to avoid ABI mismatch.
CMake ≥ 3.14	Build system	3.31 used. Generator: <code>MinGW Makefiles</code> .
Node.js	Runs <code>npx @sentry/cli</code>	So you don't have to install <code>sentry-cli</code> globally.
Sentry account + project	Provides the DSN & receives events	Create a project of platform <i>Native</i> (C/C++).
<code>sentry-native</code> source	The SDK you compile & link	github.com/getsentry/sentry-native (vendor it into your repo).
A Chromium browser	Session Replay + viewing	Edge/Chrome. Disable ad-blockers for localhost or replays get blocked.

Values you will need (record yours)

Value	Where it comes from	Example (this project)
DSN	Sentry → Project → Settings → Client Keys	<code>https://<key>@o<org-id>.ingest.us.sentry.io/<project-id></code>
Org slug	Sentry URL / org settings	<code>daanish</code>
Project id (numeric)	Last path segment of the DSN	<code>4511585577992192</code>
Auth token	Sentry → Settings → Auth Tokens (org token)	scope <code>org:ci</code> — keep secret
Release name	You choose; set in code	<code>my-project-name@2.3.12</code>

3 · Step 1 — Build the Sentry Native SDK

Vendor the SDK into your repo and compile it once. This project used the **inproc** backend (simplest; no extra handler executable) with the default **winhttp** transport, as a **shared** library.

```
# From your project root - clone WITH submodules (crashpad/breakpad deps)
git clone --recurse-submodules https://github.com/getsentry/sentry-native.git

# Configure + build the SDK (MinGW, inproc backend, shared lib, Release)
cmake -S sentry-native -B sentry-native/build -G "MinGW Makefiles" ^
      -DCMAKE_BUILD_TYPE=Release ^
      -DSENTRY_BACKEND=inproc ^
      -DSENTRY_BUILD_SHARED_LIBS=ON
cmake --build sentry-native/build --parallel
```

Expected outputs (in `sentry-native/build/`)

- `libsentry.dll` — the runtime library (must ship next to your `.exe`).
- `libsentry.dll.a` — the import library you link against.
- Header at `sentry-native/include/sentry.h`.

Option	Choices	Guidance
<code>SENTRY_BACKEND</code>	<code>inproc</code> · <code>crashpad</code> · <code>breakpad</code>	inproc = self-contained, used here. crashpad = most robust on Windows but ships a separate <code>crashpad_handler.exe</code> you must copy next to your app.
<code>SENTRY_TRANSPORT</code>	<code>winhttp</code> (Win default) · <code>curl</code> · <code>none</code>	Leave default on Windows.
<code>SENTRY_BUILD_SHARED_LIBS</code>	<code>ON</code> / <code>OFF</code>	<code>ON</code> here → <code>libsentry.dll</code> . <code>OFF</code> gives a static lib (no DLL to ship).

4 · Step 2 — Wire Sentry into your CMake build

Three things: add the include path, add the link directory, link the `sentry` library — plus the **critical compile/link flags**.

```
# Headers + link dir for the SDK you built in Step 1
include_directories(sentry-native/include)
link_directories(${CMAKE_SOURCE_DIR}/sentry-native/build)

add_executable(http_server ${SOURCES})

# Link the SDK. On Windows also link ws2_32 (winsock used by winhttp/sockets).
if(WIN32)
    target_link_libraries(http_server ws2_32 sentry)
else()
    find_package(Threads REQUIRED)
    target_link_libraries(http_server Threads::Threads sentry)
endif()

# CRITICAL FLAGS for usable Sentry traces (GCC/MinGW):
target_compile_options(http_server PRIVATE -Wall -Wextra -Wpedantic -g -O0)
target_link_options(http_server PRIVATE -Wl,--build-id)
```

Why these flags matter — do not skip:

- `-g` — emits DWARF debug info. **Without it there is nothing to symbolicate.**
- `-O0` — disables optimization so each machine instruction maps to the **exact** source line. With `-O2/-O3` the compiler inlines & reorders code, and Sentry highlights the **wrong line** (e.g. the crash line is misattributed several lines away). The trailing `-O0` overrides any build-type `-O2/-O3`.
- `-Wl,--build-id` — stamps a stable build-id so Sentry can reliably match the uploaded debug file.

Optional: auto-upload symbols on every build

Add this block so a successful build also pushes the debug files (Step 5) — opt-in, so normal builds are unaffected.

```
option(SENTRY_UPLOAD_SYMBOLS "Upload debug-information files to Sentry after build" OFF)

if(SENTRY_UPLOAD_SYMBOLS)
    find_program(SENTRY_CLI sentry-cli)
    if(NOT SENTRY_CLI)
        message(WARNING "SENTRY_UPLOAD_SYMBOLS=ON but sentry-cli not found; skipping.")
    else()
        set(SENTRY_DLL "${CMAKE_SOURCE_DIR}/sentry-native/build/libsentry.dll")
        add_custom_command(TARGET http_server POST_BUILD
            COMMAND ${SENTRY_CLI} debug-files upload --include-sources
                "${<TARGET_FILE:http_server>} ${SENTRY_DLL}"
            COMMENT "Uploading debug-information files to Sentry" VERBATIM)
    endif()
endif()
```

Enable with `cmake -DSENTRY_UPLOAD_SYMBOLS=ON ...`; reads `SENTRY_AUTH_TOKEN`, `SENTRY_ORG`, `SENTRY_PROJECT` from the environment.

5 · Step 3 — Initialise Sentry in code

One small translation unit (e.g. `sentry.cpp`) that initialises the SDK, called once at startup.

```
// sentry.cpp
#include <sentry.h>

void sentry_func(void) {
    sentry_options_t *options = sentry_options_new();
    sentry_options_set_dsn(options, "https://<key>@o<org>.ingest.us.sentry.io/<project>");
    sentry_options_set_database_path(options, ".sentry-native"); // crash DB on disk
    sentry_options_set_release(options, "my-project-name@2.3.12"); // must match DIF upload
    sentry_options_set_debug(options, 1); // prints SDK logs to stderr - keep on while wiring
    up
    sentry_init(options);
}
```

```
// main.cpp - bracket your program with init / close
int main() {
    sentry_func(); // start monitoring (installs crash handlers)
    // ... run your application ...
    sentry_close(); // flush queued events & shut down cleanly
    return 0;
}
```

Runtime requirement: `libsentry.dll` must sit next to your `.exe` (or be on `PATH`). If it can't be found, the app fails to start and you get no events at all.

How crashes get delivered (inproc): when the process crashes, the in-process handler writes the event into the `.sentry-native` database and the process exits. The event is uploaded on the **next launch**. So the test loop is: trigger crash → process dies → **start the app again** → event appears.

6 · Step 4 — Capturing errors (three modes)

Mode A — Automatic native crashes

Once `sentry_init` has run, hard crashes (access violations, `abort`) are captured automatically. No code needed. Example fault used for testing:

```
volatile int* ptr = nullptr;
*ptr = 42; // EXCEPTION_ACCESS_VIOLATION → captured automatically
```

Mode B — Caught C++ exceptions → proper Error events

An **exception event** carries a *type* + *value* and an attached stack trace, so it shows up as a first-class error. Capture the stack and attach it:

```
void report_to_sentry(const char* type, const char* value) {
    sentry_value_t event = sentry_value_new_event();
    sentry_value_t exc    = sentry_value_new_exception(type, value);
    sentry_value_set_stacktrace(exc, nullptr, 0); // nullptr => capture current stack
    sentry_event_add_exception(event, exc);
    sentry_capture_event(event);
}
// usage inside catch: report_to_sentry(typeid(e).name(), e.what());
```

Mode C — Manual events (logs / custom errors)

The trap that wasted the most time: `sentry_value_new_message_event(...)` does **NOT** include a stack trace. A bare message event shows up as "(No error message)" with no frames and **nothing to symbolicate**. You must attach a stack trace yourself.

Modern (non-deprecated) way — build a stacktrace, hang it on a thread (or an exception), add to the event:

```
// Capture the stack BEFORE building the event so it reflects the real call site,
// not the Sentry SDK internals.
sentry_value_t stack = sentry_value_new_stacktrace(nullptr, 0);

sentry_value_t event = sentry_value_new_event();
sentry_value_t exc    = sentry_value_new_exception("Tog on", "Toggle switched ON");
sentry_value_set_by_key(exc, "stacktrace", stack); // attach pre-captured stack
sentry_event_add_exception(event, exc);
sentry_capture_event(event);
```

Attaching to a *thread* instead: `sentry_value_new_thread(id, "main")` → `sentry_value_set_by_key(thread, "stacktrace", stack)` → `sentry_event_add_thread(event, thread)`. The older one-liner `sentry_event_value_add_stacktrace(event, nullptr, 0)` still works but is deprecated.

Event kind	API	Appears as	Has trace?
Native crash	automatic	Fatal / AccessViolation	Yes (auto)
Exception event	<code>new_exception</code> + <code>add_exception</code>	Error w/ type:value	Yes — if you attach one
Message event	<code>new_message_event</code>	Plain message	No unless you attach one

7 · Step 5 — Upload debug symbols MAKE-OR-BREAK

This is what turns blank traces into your source code. Do it after every build.

5.1 — Create an auth token

Sentry → **Settings** → **Auth Tokens** → create an **organization** token. Scope `org:ci` is enough to upload (it cannot list/read issues — that's fine).

5.2 — Run the upload

```
# Set secrets in the shell (PowerShell shown). Never hard-code the token.
$env:SENTRY_AUTH_TOKEN = "<YOUR_TOKEN>"

# Upload the EXE (+ the SDK DLL) with source context. -p accepts the numeric project id.
npx --yes @sentry/cli@latest debug-files upload --include-sources ^
-o daanish -p 4511585577992192 ^
"build/http_server.exe" "sentry-native/build/lib Sentry.dll"
```

Expected output

```
> Found 1 debug information file
> Resolved source code for 1 debug information file
> Uploaded 2 missing debug information files
  UPLOADED 7e8f49de-...-1 (http_server.exe; x86_64 executable)
  UPLOADED 7e8f49de-...-1 (http_server.exe; x86_64 sources)
```

`--include-sources` is what makes the actual code lines appear in the stack trace (it bundles your `.cpp/.hpp` files keyed to the debug-id). Without it you get function:line but no code snippet.

5.3 — Verify

```
npx @sentry/cli@latest debug-files check "build/http_server.exe"
# Look for:  Debug ID: ... Usable: yes
# The Debug ID must match what you uploaded (and the binary that actually runs).
```

Re-upload after EVERY rebuild. The debug-id changes on each compile; an event from a new binary won't match an old upload, and you'll silently fall back to blank traces. Automate it with the CMake `SENTRY_UPLOAD_SYMBOLS` option from Step 2, or just rerun 5.2.

8 · Step 6 — Session Replay (the "video")

Replay is browser-only. The Native SDK cannot record a session. A replay attaches only to errors the **browser** reports — a backend C++ error will **never** show a video. To get a replay tied to an error, the browser page must (a) load the Sentry Browser SDK and (b) capture the error itself.

In the HTML page your server serves, load the **single combined bundle** and init:


```

<!-- ONE bundle: core + tracing + replay. Loading bundle.min.js and
      replay.min.js separately can leave Sentry.replayIntegration undefined,
      which makes Sentry.init() throw and silently disables everything. -->
<script src="https://browser.sentry-cdn.com/8.9.0/bundle.tracing.replay.min.js"
crossorigin="anonymous"></script>
<script>
  Sentry.init({
    dsn: "<same DSN as the backend>",
    integrations: [ Sentry.browserTracingIntegration(),
Sentry.replayIntegration({maskAllText:false, blockAllMedia:false}) ],
    tracesSampleRate: 1.0,
    replaysSessionSampleRate: 1.0, // record 100% of sessions (the video)
    replaysOnErrorSampleRate: 1.0,
  });
</script>

```

Then, where the user acts, report from the browser so the active replay links to it:

```
Sentry.captureException(new Error(isOn ? 'Tog on' : 'Tog off'));
```

View replays under Sentry's **Replays** tab (sidebar), or on a browser issue's **Replays** section. Disable ad-blockers/Brave shields for `localhost` or the replay never sends.

9 · Step 7 — Verify end-to-end

Handy test routes (one per event kind) and what each should produce in Sentry:

Trigger	Code	Expect in Sentry
<code>/sentry-crash</code>	<code>*ptr = 42; (null write)</code>	Fatal · AccessViolation; top in-app frame = the exact <code>*ptr = 42</code> line. (Process dies — restart to flush.)
<code>/sentry-exception</code>	<code>throw std::runtime_error(...)</code>	Error, caught & reported with the throw-site stack.
Manual event	exception/message + attached stack	Error with your type:value and a symbolicated trace.
Browser action	<code>Sentry.captureException</code>	Browser error with a Replay video attached.

Success looks like: the top frame reads e.g. `operator() (main.cpp:164)` and expanding it shows your source with the faulting line highlighted. If the line is right and the code shows — both halves work.

Run exactly one instance of the server. Extra copies fail to bind the port, crash on the bind error, and flood Sentry with noise.

10 · Troubleshooting matrix (the gotchas)

Symptom	Root cause	Fix
Issue appears but trace is blank / <code><unknown> +0xNNN</code>	Debug files not uploaded for this binary	Run Step 5 upload; verify with <code>debug-files check</code> .
Trace shows <code>fn (file:line)</code> but no code snippet	Uploaded without sources	Re-upload with <code>--include-sources</code> .
Wrong line highlighted / frames point at unrelated code	Optimization (<code>-O2/-O3</code>) inlined & reordered	Build with <code>-O0 -g</code> ; re-upload.
Event shows "(No error message)", no frames	Message event with no stack attached	Attach <code>new_stacktrace</code> via thread/exception, or send an exception event.
Trace points at the Sentry call line, not the real context	Stack captured after building the event	Call <code>sentry_value_new_stacktrace()</code> first , then attach.
Symbols worked yesterday, blank today	Rebuilt → new debug-id	Re-upload after every build (automate it).
No "video" on the error	Backend error, or browser SDK not initialised	Replay is browser-only: use the combined bundle & <code>Sentry.captureException</code> in the page.
App won't start / no events at all	<code>libsentry.dll</code> not found	Copy the DLL next to the <code>.exe</code> or add to <code>PATH</code> .
<code>cannot open output file ...exe: Permission denied</code> (link)	The app is still running and locks the exe	Stop all instances before rebuilding.
403 when listing projects with the token	<code>org:ci</code> can upload but not read	Normal — use the numeric project id with <code>-p</code> ; upload still works.

11 · Security & housekeeping

- **Never commit the auth token.** Keep it in an env var / secret store. If it leaks, **rotate it** (Sentry → Settings → Auth Tokens → revoke).
- The **DSN is not a strong secret** (it ships in the browser anyway), but env-var it for cleanliness.
- `.gitignore` at least: `build/`, `.sentry-native/`, any `*.src.zip`.
- Keep `sentry_options_set_debug(1)` on while wiring up, then turn it off for production.

12 · Optional — link stack frames to your GitHub repo

To get "Open in GitHub" links that jump to the exact file/line/commit (Sentry "Stack Trace Linking" + "Suspect Commits"):

1. Put the project on GitHub (`git init` → commit → push).
2. Sentry → **Settings** → **Integrations** → **GitHub** → install, then add the repository.
3. Project → **Stack Trace Linking** (Code Mappings): map the stack-trace path root (e.g. `C:\path\to\project\`) → the repo source root → repo + default branch.

4. Associate releases with commits so links resolve to the right SHA:

```
npx @sentry/cli releases new "my-project-name@2.3.12"
npx @sentry/cli releases set-commits "my-project-name@2.3.12" --auto
npx @sentry/cli releases finalize "my-project-name@2.3.12"
```

Use the **same** release string here as in `sentry_options_set_release(...)`.

Appendix A — Command cheat sheet

```
# 1) Build the SDK (once)
cmake -S sentry-native -B sentry-native/build -G "MinGW Makefiles" -DSENTRY_BACKEND=inproc -
DSENTRY_BUILD_SHARED_LIBS=ON -DCMAKE_BUILD_TYPE=Release
cmake --build sentry-native/build --parallel

# 2) Configure + build your app (debug info, no optimization)
cmake -S . -B build -G "MinGW Makefiles"
cmake --build build

# 3) Upload symbols + sources (RE-RUN AFTER EVERY BUILD)
$env:SENTRY_AUTH_TOKEN = "<token>"
npx --yes @sentry/cli@latest debug-files upload --include-sources -o <org> -p <project-id>
"build/http_server.exe" "sentry-native/build/libentry.dll"

# 3b) Or build + auto-upload in one shot
$env:SENTRY_AUTH_TOKEN="<token>"; $env:SENTRY_ORG="<org>"; $env:SENTRY_PROJECT="<project-id>"
cmake -S . -B build -DSENTRY_UPLOAD_SYMBOLS=ON ; cmake --build build

# 4) Verify a binary's debug-id
npx @sentry/cli@latest debug-files check "build/http_server.exe"

# 5) Run (one instance, from project root so it finds static/ and writes .sentry-native/)
build/http_server.exe
```

Appendix B — Final file listings (reference project)

CMakeLists.txt (key parts)

```

cmake_minimum_required(VERSION 3.14)
project(http_server CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)
include_directories(sentry-native/include)
link_directories(${CMAKE_SOURCE_DIR}/sentry-native/build)

add_executable(http_server ${SOURCES})

if(WIN32)
    target_link_libraries(http_server ws2_32 sentry)
else()
    find_package(Threads REQUIRED)
    target_link_libraries(http_server Threads::Threads sentry)
endif()

target_compile_options(http_server PRIVATE -Wall -Wextra -Wpedantic -g -O0)
target_link_options(http_server PRIVATE -Wl,--build-id)

option(SENTRY_UPLOAD_SYMBOLS "Upload DIFs to Sentry after build" OFF)
# ... POST_BUILD sentry-cli block (see §4) ...

```

sentry.cpp

```

#include <sentry.h>
void sentry_func(void) {
    sentry_options_t *options = sentry_options_new();
    sentry_options_set_dsn(options, "<YOUR DSN>");
    sentry_options_set_database_path(options, ".sentry-native");
    sentry_options_set_release(options, "my-project-name@2.3.12");
    sentry_options_set_debug(options, 1);
    sentry_init(options);
}

```

— End of manual. Generated from the [sentryc++](#) reference project. Replace all example identifiers (DSN, org [daanish](#), project [4511585577992192](#), release [my-project-name@2.3.12](#)) with your own.